



ESCUELA DE INGENIERÍA DE FUENLABRADA

CIENCIA E INGENIERÍA DE DATOS

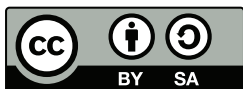
TRABAJO FIN DE GRADO

UN TÍTULO DE PROYECTO LARGO
EN DOS LÍNEAS

Autor : Pablo Pardo Gutiérrez

Tutor : Dr. José Felipe Ortega Soto

Curso Académico 2025/2026



© 2026 Nombre y apellidos . Algunos derechos reservados.

El código fuente de este Trabajo Fin de Grado está disponible en:
[enlacegithubaqui](#)

Este trabajo se entrega bajo la licencia Creative Commons
Reconocimiento-CompartirIgual (by-sa). Para obtener
la licencia completa, véase:
<https://creativecommons.org/licenses/by-sa/4.0/deed.es>.

Primera impresión, 23 de marzo de 2026

Trabajo Fin de Grado

Título del Trabajo con Letras Capitales para Sustantivos y Adjetivos

Autor : Pablo Pardo Gutiérrez

Tutor : Dr. José Felipe Ortega Soto

La defensa del presente Proyecto Fin de Grado se realizó el día 3 de
de 2026, siendo calificada por el siguiente tribunal:

Presidente:

Secretario:

Vocal:

y habiendo obtenido la siguiente calificación:

Calificación:

Móstoles/Fuenlabrada, a de de 2026

*dedicacion
aqui
gracias*

Agradecimientos

Aquí vienen los agradecimientos...

Hay más espacio para explayarse y explicar a quién agradeces su apoyo o ayuda para haber acabado el proyecto: familia, pareja, amigos, compañeros de clase...

También hay quien, en algunos casos, hasta agradecer a su tutor o tutores del proyecto la ayuda prestada...

Resumen

Aquí viene un resumen del proyecto. Ha de constar de tres o cuatro párrafos, donde se presente de manera clara y concisa de qué va el proyecto. Han de quedar respondidas las siguientes preguntas:

- ¿De qué va este proyecto? ¿Cuál es su objetivo principal?
- ¿Cómo se ha realizado? ¿Qué tecnologías están involucradas?
- ¿En qué contexto se ha realizado el proyecto? ¿Es un proyecto dentro de un marco general?

Lo mejor es escribir el resumen al final.

Summary

Here comes a translation of the “Resumen” into English. Please, double check it for correct grammar and spelling. As it is the translation of the “Resumen”, which is supposed to be written at the end, this as well should be filled out just before submitting.

Índice general

1	Introducción	1
1.0.1	Motivación	1
1.0.2	Caso de Estudio: La ciudad de Melbourne	2
1.1	Estado del Arte	3
1.1.1	Consolidación del IoT en la Gestión Inteligente del Aparcamiento	3
1.1.2	Evolución hacia el <i>Modern Data Stack</i> en Entornos IoT	4
1.1.3	Modelado de Series Temporales y el Reto de los Datos Parcialmente Observados	4
1.2	Objetivos del proyecto	5
1.2.1	Objetivo general	5
1.2.2	Objetivos específicos	5
1.3	Planificación temporal	6
1.3.1	Diagrama de Gantt	7
1.4	Estructura de la memoria	8
2	Tecnologías	9
2.1	Lenguaje Base y Gestión de Entorno	9
2.1.1	Python	9
2.1.2	Gestor de dependencias: <i>uv</i>	9
2.2	Arquitectura de Ingesta y Almacenamiento	10
2.2.1	Apache Airflow	10
2.2.2	DuckDB	11
2.3	Ciencia de Datos y <i>Deep Learning</i>	11
2.3.1	PyPOTS	11
2.4	Visualización	12
2.4.1	Grafana y Herramientas Geoespaciales	12
2.4.2	Quarto	12
3	Diseño e implementación	14
3.1	Arquitectura general	14

3.2	Origen y Comprensión de los Datos	16
3.2.1	Justificación del <i>dataset</i> histórico. Extracción automatizada	16
3.2.2	Estructura del conjunto de datos	17
3.3	Análisis Exploratorio de Datos (EDA)	17
3.3.1	Transformación y tipificado de variables	19
3.3.2	Ingeniería de características (<i>Feature Engineering</i>) . . .	19
3.3.3	Análisis descriptivo y visualización	19
3.3.4	Selección de características	21
3.4	Adaptación temporal para <i>Deep Learning</i>	
	22	
3.4.1	Regularización de la serie (<i>Grid</i>)	22
3.4.2	Codificación categórica posicional (<i>One-Hot Encoding</i>)	22
4	Experimentos y validación	24
4.1	Incorporación de código en la memoria	24
4.1.1	Fuentes monoespaciadas	25
4.2	Contenido matemático y ecuaciones	26
4.2.1	Ecuaciones en múltiples líneas y elementos alineados . .	27
4.2.2	Teoremas y conjuntos	28
5	Conclusiones y trabajos futuros	29
5.1	Consecución de objetivos	29
5.2	Aplicación de lo aprendido	29
5.3	Lecciones aprendidas	30
5.4	Trabajos futuros	30
A	Contenido adicional	31
A.1	Dimensiones de página	31
	Referencias	35

Índice de figuras

1.1	Diagrama de Gantt ilustrando la temporalización de las fases del proyecto y sus respectivas tareas a lo largo de seis meses.	7
3.1	Arquitectura general del pipeline del sistema.	15
3.2	Distribución de la variable objetivo (<i>VehiclePresent</i>).	20
3.3	Distribución del volumen de eventos de estacionamiento según la hora del día.	20
3.4	Top de vías o zonas con mayor densidad de registros de aparcamiento (<i>hotspots</i>).	21
3.5	Transformación de variables temporales mediante <i>One-Hot Encoding</i> en el <i>dataset</i> final preprocesado.	22

Índice de tablas

List of Listings

4.1	Lectura de un fichero *.csv y tipado de datos.	25
-----	--	----

1 Introducción

El despliegue de redes IoT (Internet de las Cosas) representa la piedra angular en la transición de las urbes tradicionales hacia las *Smart Cities* (o Ciudades Inteligentes). Históricamente, la planificación urbana y la gestión del tráfico se basaban en estudios estadísticos puntuales que quedaban obsoletos rápidamente. Sin embargo, la integración del IoT ha transformado la infraestructura física de las ciudades en un ecosistema digital dinámico e interconectado.

En este nuevo paradigma, los sensores IoT actúan como el “sistema nervioso” de la ciudad. Estos dispositivos, integrados en el asfalto, semáforos, farolas o contenedores, son capaces de capturar, procesar y transmitir parámetros físicos en tiempo real. La verdadera disrupción tecnológica del IoT urbano no reside únicamente en la recolección aislada de datos, sino en la capacidad de generar un flujo continuo de información a gran escala (*Big Data*). Esto permite a las administraciones públicas y a los sistemas automatizados pasar de una gobernanza reactiva, que actúa cuando el problema ya se ha manifestado, a una proactiva y predictiva, lo que permite maximizar la eficiencia y anticipar casuísticas de interés.

1.0.1 Motivación

El impacto del IoT en la movilidad urbana es especialmente crítico. En las últimas décadas, el crecimiento demográfico y la rápida urbanización han transformado la congestión vehicular en uno de los mayores desafíos para las administraciones públicas, generando un impacto negativo tanto en el medio ambiente, por el aumento de emisiones de gases de efecto invernadero, como en la calidad de vida de los ciudadanos. Diversos estudios urbanísticos señalan que el tráfico de agitación —aquel provocado por vehículos que circulan a baja velocidad buscando activamente aparcamiento— es responsable de hasta un 30% de la congestión en los centros urbanos, lo que representa una fuente masiva de contaminación atmosférica y acústica [7].

Para mitigar este problema, la instalación de sensores electromagnéticos o de infrarrojos en las plazas de aparcamiento en superficie permite monitorizar la ocupación exacta en tiempo real. No obstante, la implementación práctica de estas redes de sensores se enfrenta a un desafío técnico ineludible: la volatilidad y la pérdida de integridad de los datos. Al tratarse de dispositivos físicos distribuidos en un entorno hostil (la vía pública), los sensores están sujetos a fallos de conectividad, latencia de red, averías por factores climáticos o agotamiento de baterías [4]. Consecuentemente, las series temporales de datos quedan discontinuas y plagadas de valores faltantes, lo que distorsiona severamente cualquier análisis visual o intento de toma de decisiones informada.

Por consiguiente, la motivación principal de este proyecto surge de la necesidad de construir un ecosistema integral y resiliente que aborde el ciclo de vida completo del dato urbano. No es suficiente con instalar sensores; es imperativo diseñar una arquitectura de ingeniería de datos robusta y automatizada que ingeste y almacene esta información en tiempo real. Asimismo, es crucial la aplicación de técnicas vanguardistas de *Deep Learning* capaces de comprender los patrones espaciotemporales subyacentes para imputar y reconstruir la información faltante cuando el hardware falla. Solo garantizando esta calidad e integridad técnica, el flujo incesante de datos del IoT puede ser explotado eficazmente, transformando los datos crudos en valor real y democratizado para la ciudadanía.

1.0.2 Caso de Estudio: La ciudad de Melbourne

Para materializar y validar la arquitectura propuesta, este proyecto toma como escenario de aplicación la red de aparcamientos en superficie de la ciudad de Melbourne (Australia). La elección de esta urbe responde a criterios técnicos y de disponibilidad de datos que la convierten en un entorno de pruebas óptimo para el desarrollo del proyecto.

En primer lugar, el gobierno local de Melbourne es pionero a nivel mundial en políticas de *Open Data*, al ofrecer acceso público y documentado a su infraestructura IoT. A diferencia de otras grandes ciudades que únicamente proporcionan datos agregados de aparcamientos subterráneos, Melbourne ofrece una granularidad a nivel de plaza individual.

En segundo lugar, la plataforma proporciona una dualidad de acceso crítica para este proyecto. Por un lado, un repositorio histórico masivo, como el *dataset* del año 2019 escogido para este proyecto, el cual es indispensable para la fase de entrenamiento de modelos de *Deep Learning*. Por otro lado, una API REST que

emite el estado de los sensores en tiempo real, lo cual habilita la implementación de un *pipeline* de orquestación continua.

Finalmente, al tratarse de una red de sensores físicos expuesta a condiciones urbanas reales durante años, el conjunto de datos de Melbourne no es un entorno estéril o simulado. Contiene intrínsecamente las imperfecciones, latencias, fallos de batería y valores faltantes descritos en la motivación de este trabajo. Esta naturaleza imperfecta es precisamente lo que legitima y hace necesaria la aplicación de herramientas de imputación avanzadas como PyPOTS, lo que permite demostrar el valor de la solución propuesta en un escenario de extrema fidelidad con el mundo real.

1.1 Estado del Arte

1.1.1 Consolidación del IoT en la Gestión Inteligente del Aparcamiento

En la última década, el Internet de las Cosas (IoT) ha superado su fase experimental para consolidarse como la infraestructura subyacente de las *Smart Cities*. Este avance ha sido impulsado por el despliegue de redes de bajo consumo y largo alcance (LPWAN) y la transición hacia el *Edge Computing*, lo cual hace posible sensorizar masivamente infraestructuras críticas urbanas [1].

Dentro de la movilidad urbana, el *Smart Parking* destaca como uno de los campos con mayor madurez tecnológica. La información generada por los sensores ha dejado de ser puramente descriptiva para integrarse en tres grandes casos de uso operativos: (I) sistemas de guiado en tiempo real para reducir el tráfico de agitación; (II) tarificación dinámica (*Dynamic Pricing*) basada en la demanda espacial; y (III) control telemático de infracciones (*Enforcement*). Sin embargo, el verdadero potencial de estos sistemas requiere trascender la monitorización actual hacia la analítica predictiva, lo cual exige infraestructuras de procesamiento de datos altamente eficientes [5].

1.1.2 Evolución hacia el *Modern Data Stack* en Entornos IoT

El procesamiento del ingente volumen de datos espaciotemporales generado por las redes ha evidenciado las limitaciones de las arquitecturas monolíticas tradicionales ETL (*Extract, Transform, Load*). Históricamente, la gestión de estos flujos requería bases de datos pesadas y costosos clústeres computacionales.

Actualmente, el estado del arte a nivel industrial y de investigación converge hacia el paradigma del *Modern Data Stack* (MDS). Esta evolución se caracteriza por la adopción de herramientas ágiles, desacopladas y orientadas al código (*Data-as-Code*). En la fase de orquestación, planificadores dinámicos como Apache Airflow se han estandarizado para gestionar dependencias complejas de ingesta. Simultáneamente, el paradigma de almacenamiento analítico ha experimentado una disrupción con la aparición de motores OLAP (procesamiento analítico en línea) en proceso (*in-process*), entre los que DuckDB destaca como su máximo exponente actual. Estos sistemas permiten ejecutar consultas vectorizadas sobre datos IoT directamente en entornos locales o *edge*, superando la latencia inherente a las bases de datos cliente-servidor tradicionales [6].

1.1.3 Modelado de Series Temporales y el Reto de los Datos Parcialmente Observados

Pese a contar con infraestructuras de datos robustas, la extracción de valor predictivo se topa con el desafío técnico expuesto en la motivación: la naturaleza intermitente de la señal física. Los algoritmos clásicos de *Machine Learning* y las redes neuronales estándar tienen un comportamiento errático ante la presencia de huecos o valores nulos en las secuencias temporales.

En los últimos años, el análisis secuencial ha dado un salto cualitativo gracias a la adaptación de arquitecturas de *Deep Learning* avanzadas (*Transformers*) al dominio temporal. No obstante, la vanguardia investigadora actual no se centra únicamente en la predicción pura, sino en el modelado de Datos Temporales Parcialmente Observados (*Partially Observed Time Series* o POTS).

Para abordar esta problemática, la comunidad científica ha transitado desde heurísticas simples hacia *frameworks* de aprendizaje profundo especializados en

imputación. Librerías recientes de código abierto, como PyPOTS, se han posicionado como el estándar en investigación [3]. Estas herramientas implementan modelos capaces de inferir el dato faltante basándose en el contexto espacio-temporal global de la serie, para así alimentar los posteriores cuadros de mando analíticos y sistemas de información geográfica con información íntegra y fiable.

1.2 Objetivos del proyecto

1.2.1 Objetivo general

Desarrollar un sistema integral de procesamiento y modelado para predecir la disponibilidad de plazas de aparcamiento en superficie, gestionando y reconstruyendo series temporales espacialmente distribuidas provenientes de una red de datos de sensores IoT en tiempo real.

1.2.2 Objetivos específicos

Para alcanzar el objetivo general, el proyecto se desglosa en las siguientes metas técnicas secuenciales:

- **Realizar un Análisis Exploratorio de Datos históricos (EDA).** Analizar el conjunto de datos históricos masivos para identificar distribuciones, patrones de estacionalidad en la movilidad urbana y anomalías en los tiempos de estacionamiento.
- **Preprocesar el *dataset* y entrenar el modelo.** Acondicionar el conjunto de datos y posteriormente entrenar el modelo de PyPOTS, para que el algoritmo aprenda los patrones subyacentes de ocupación y la estructura de la serie temporal.
- **Desplegar la infraestructura de ingesta y almacenamiento continuo.** Orquestar un *pipeline* automatizado utilizando Apache Airflow para consumir datos en tiempo real de la API de Melbourne durante un periodo ininterrumpido de un mes, y almacenar el histórico resultante en una base de datos analítica centralizada (DuckDB).

- **Imputar datos faltantes en la serie temporal actual.** Aplicar el modelo PyPOTS previamente entrenado sobre los datos reales recopilados durante el mes de monitorización con Airflow, con el fin de reconstruir la serie temporal e imputar los valores faltantes (*missing data*) generados por los fallos inherentes de la sensórica física.
- **Desarrollar un *dashboard* analítico.** Construir un cuadro de mando interactivo conectando Grafana directamente a DuckDB para la monitorización de datos y métricas.
- **Construir un visualizador geoespacial.** Visualizar la red de aparcamientos categorizada por su proximidad a Puntos de Interés (POIs) en un mapa 2D.
- **Divulgar los resultados mediante un entorno web estructurado.** Diseñar y desplegar una página web interactiva con Quarto, dividida en tres secciones principales, que actúe como portfolio visual para exponer la metodología, la arquitectura de datos y los resultados del proyecto de manera accesible.

1.3 Planificación temporal

Para garantizar la consecución de los objetivos descritos y asegurar la viabilidad técnica, el ciclo de vida del proyecto se ha estructurado en cuatro fases de desarrollo iterativo e incremental.

- **Fase 1: Conceptualización y despliegue de infraestructura.** En esta etapa inicial se llevó a cabo la revisión bibliográfica del estado del arte y el diseño teórico de la arquitectura. Seguidamente, se configuraron los entornos virtualizados y se implementaron los flujos de orquestación con Apache Airflow, y se estableció la conexión temprana con la API de Melbourne para posteriormente iniciar la ventana de ingesta y almacenamiento continuo en DuckDB.
- **Fase 2: Procesamiento histórico y *Feature Engineering*.** Esta fase se centró en la adecuación del conjunto de datos masivo correspondiente al año 2019. Se ejecutaron labores de limpieza, integración de metadatos geoespaciales y, fundamentalmente, el remuestreo (*resampling*) de los eventos vehiculares asíncronos. Esto permitió estandarizar la información y construir las matrices temporales con valores nulos explícitos requeridas para el modelado.

- **Fase 3: Modelado predictivo y visualizaciones.** Con los datos estructurados, se procedió al diseño, entrenamiento y ajuste de los modelos de *Deep Learning* de PyPOTS. En paralelo, se construyó el ecosistema de visualización conectando Grafana a la base de datos analítica y desarrollando mapas interactivos para la representación espacial de las plazas y su relación con lugares próximos relevantes.
- **Fase 4: Validación y Cierre.** La etapa final consistió en la validación del ecosistema, aplicando el modelo entrenado sobre los datos recopilados en tiempo real para ejecutar la imputación de valores faltantes. Finalmente, se desplegó el entorno web divulgativo mediante Quarto y se procedió a la extracción de conclusiones y revisión exhaustiva de la presente memoria académica.

El proyecto se ha desarrollado durante el periodo comprendido entre los meses de enero y junio. El avance técnico se ha abordado mediante una metodología de trabajo continua, integrando la ejecución práctica de las fases descritas con la redacción y revisión progresiva de la presente memoria académica.

1.3.1 Diagrama de Gantt

Para ilustrar visualmente la planificación temporal detallada en la sección anterior, se ha elaborado el diagrama de Gantt correspondiente al ciclo de vida del proyecto. Esta representación gráfica permite observar la duración y secuenciación de las tareas específicas que componen cada una de las cuatro fases principales.

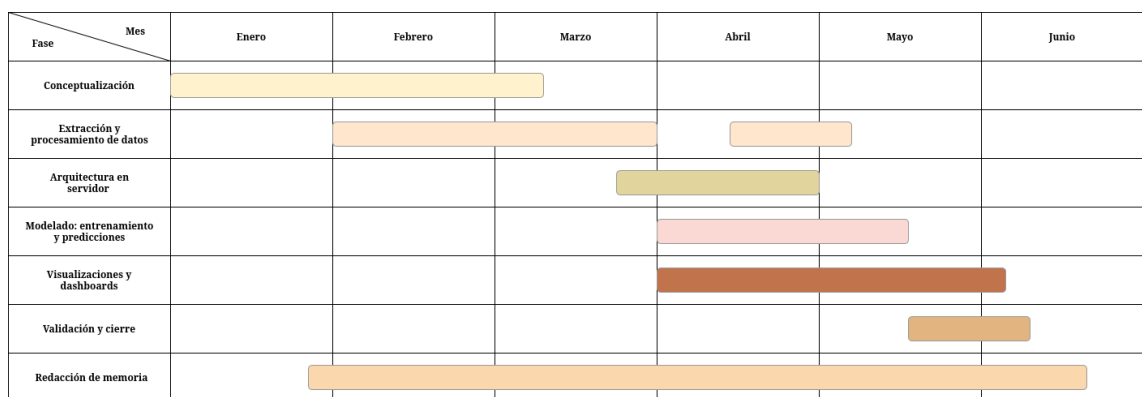


Figura 1.1: Diagrama de Gantt ilustrando la temporalización de las fases del proyecto y sus respectivas tareas a lo largo de seis meses.

1.4 Estructura de la memoria

El resto del presente documento se estructura en cuatro capítulos adicionales, organizados de manera secuencial para reflejar el ciclo de vida del proyecto:

- El **capítulo 2** (Tecnologías empleadas) detalla las herramientas, lenguajes y *frameworks* utilizados para la consecución del proyecto. Se justifica la elección de cada pieza de *software* dentro del paradigma del *Modern Data Stack*.
- El **capítulo 3** (Diseño e Implementación) constituye el núcleo técnico de la memoria. En él se describe exhaustivamente el desarrollo del *pipeline* de datos, los procesos de limpieza y *resampling* del conjunto histórico, así como el entrenamiento del modelo de predicción. Asimismo, se detallan el diseño arquitectónico de la orquestación en tiempo real y la construcción del ecosistema de visualización analítica y geoespacial.
- El **capítulo 4** (Experimentación y Resultados) presenta las pruebas realizadas y los hallazgos obtenidos. Se evalúa el rendimiento del modelo de imputación espacial y temporal entrenado con los datos reales recopilados durante la ventana de monitorización, y se exponen los resultados visuales de los dashboard.
- Finalmente, el **capítulo 5** (Conclusiones) sintetiza las deducciones finales extraídas tras la finalización del trabajo. Posteriormente, se evalúa el grado de cumplimiento de los objetivos planteados y se proponen futuras líneas de investigación y desarrollo que podrían derivarse de esta base.

2 Tecnologías

Para dar respuesta a las necesidades del proyecto, se ha diseñado una arquitectura de datos robusta, escalable y reproducible. La selección de herramientas se ha fundamentado en los estándares actuales de la industria y el paradigma del *Modern Data Stack*. En este capítulo se detallan las tecnologías empleadas, así como la justificación técnica que motiva su adopción frente a otras alternativas del mercado.

2.1 Lenguaje Base y Gestión de Entorno

2.1.1 Python

Python se ha consolidado como el lenguaje de programación de referencia en el ecosistema de la Ciencia de Datos y la Inteligencia Artificial, gracias a su sintaxis legible y su vasta comunidad de desarrollo [8]. En este proyecto, Python actúa como el hilo conductor que vertebra todas las fases de la arquitectura, desde los *scripts* de extracción de datos hasta el entrenamiento de las redes neuronales.

2.1.2 Gestor de dependencias: *uv*

Una práctica ineludible en cualquier proyecto de ingeniería de *software* es garantizar la reproducibilidad del entorno virtual. Para la gestión de dependencias y paquetes, se ha adoptado *uv*, una herramienta de vanguardia desarrollada en el lenguaje Rust que ha irrumpido recientemente como sustituto de alto rendimiento para el gestor tradicional *pip*.

La justificación técnica para la adopción de *uv* radica en su eficiencia arquitectónica extrema. Al estar compilado en Rust, aprovecha los estándares modernos de empaquetado de Python (como PEP 518 y PEP 621), ya que lee las dependencias directamente desde archivos de configuración `.toml` sin necesidad

de ejecutar código de terceros, un cuello de botella histórico en *pip*. Además, *uv* rechaza paquetes mal formados y aplica reglas estrictas que ignoran formatos obsoletos, lo que se traduce en una mayor seguridad y estabilidad del entorno.

A nivel de rendimiento, *uv* descarga múltiples paquetes de manera concurrente en lugar de realizar peticiones secuenciales. Destaca especialmente su implementación de un sistema de caché global mediante enlaces duros (*hard links*); si un paquete ya ha sido instalado previamente en otro proyecto de la máquina, *uv* enlaza los archivos en lugar de duplicarlos. Esta característica, sumada al uso del algoritmo de resolución de dependencias *PubGrub* para evitar conflictos de versiones, reduce drásticamente los tiempos de instalación y el consumo de almacenamiento en disco.

2.2 Arquitectura de Ingesta y Almacenamiento

2.2.1 Apache Airflow

Apache Airflow es una plataforma de código abierto diseñada para crear, planificar y monitorizar flujos de trabajo programáticos (*workflows*). A diferencia de herramientas de planificación clásicas basadas en el sistema operativo (como *cron*), Airflow permite definir las tuberías de datos como código (*Data-as-Code*) mediante Grafos Acíclicos Dirigidos (DAGs), lo que proporciona un control granular sobre las dependencias entre tareas.

En la arquitectura de este proyecto, Airflow actúa como el orquestador principal encargado de automatizar las extracciones recurrentes de la API del Ayuntamiento de Melbourne. Se ha desplegado en un servidor virtualizado, configurado con los requisitos de seguridad y aislamiento de red pertinentes. Esta infraestructura garantiza una ejecución ininterrumpida.

Su elección frente a otros orquestadores se fundamenta en su resiliencia. Airflow gestiona de forma nativa los reintentos ante caídas o fallos de conexión (*retries*), alerta sobre cuellos de botella y proporciona una interfaz web robusta para monitorizar el estado del *pipeline* en tiempo real, lo cual es crítico para una arquitectura de *micro-batching* (procesamiento por microlotes).

2.2.2 DuckDB

DuckDB es un sistema de gestión de bases de datos relacionales en proceso (*in-process*), diseñado específicamente para cargas de trabajo de procesamiento analítico en línea (OLAP). Frente a motores tradicionales embebidos como SQLite, que almacenan la información fila por fila (ideales para transacciones puntuales), DuckDB emplea una arquitectura de almacenamiento orientada a columnas. Además, DuckDB se ejecuta dentro de la aplicación, lo cual implica que no requiere un servidor externo, ni gestión de clústeres, ni una configuración de infraestructura compleja [6].

Esta aproximación columnar, combinada con un motor de ejecución de consultas vectorizado, permite que las operaciones analíticas complejas y las agregaciones temporales masivas se resuelvan a una velocidad significativamente superior. El motor procesa bloques de datos (vectores) en una sola instrucción de la CPU, lo que minimiza la sobrecarga computacional.

Se ha integrado DuckDB en el proyecto por su sinergia absoluta con el ecosistema de Python y, muy especialmente, por su capacidad de ejecutar sentencias SQL estándar directamente sobre *DataFrames* de Pandas. Al operar en el mismo espacio de memoria que el proceso que lo invoca, DuckDB permite realizar consultas relacionales analíticas complejas (incluyendo sentencias **JOIN**, agrupaciones o funciones de ventana) al consultar directamente variables nativas de Python, sin necesidad de copiar, exportar o transferir los datos previamente a un servidor externo. Esta versatilidad híbrida, que aúna la expresividad del lenguaje SQL con la flexibilidad analítica de Pandas, lo convierte en la base de datos ideal para transformar y almacenar el flujo incesante de datos espaciotemporales procedentes de Airflow.

2.3 Ciencia de Datos y *Deep Learning*

2.3.1 PyPOTS

PyPOTS (*A Python Toolbox for Data Mining on Partially Observed Time Series*) es una librería especializada de código abierto diseñada para el modelado, clasificación y predicción de series temporales que presentan valores nulos o discontinuidades estructurales [3]. Su arquitectura unifica implementaciones de vanguardia basadas en mecanismos de atención y redes neuronales recurrentes.

En este trabajo, se utiliza PyPOTS porque permite entrenar modelos de *Deep Learning* capaces de inferir el estado de los sensores físicos estropeados (proceso de imputación) a partir de dependencias espaciotemporales complejas. A diferencia de las técnicas de interpolación matemática clásica, que solo observan los datos históricos de un sensor aislado, los modelos disponibles en PyPOTS analizan el contexto global de la red de aparcamientos.

Al aprender la dinámica subyacente de la movilidad de la ciudad para rellenar estos huecos, el modelo extrae representaciones latentes profundas del comportamiento urbano. Este aprendizaje auto-supervisado otorga a la arquitectura la capacidad no solo de reconstruir el pasado imperfecto, sino de predecir la ocupación futura con un alto grado de precisión.

2.4 Visualización

2.4.1 Grafana y Herramientas Geoespaciales

Para la monitorización visual de la información procesada, se ha optado por Grafana, una plataforma de código abierto líder en observabilidad y *Business Intelligence* (BI). Grafana destaca por su capacidad para conectarse a múltiples fuentes de datos de forma nativa. En este proyecto, se alimenta directamente del motor analítico DuckDB, lo que permite generar cuadros de mando (*dashboards*) interactivos que reflejan métricas de interés sobre los datos de ocupación.

En el plano del análisis espacial, se han incorporado librerías de mapeo interactivas para Python, como Folium. Esta herramienta actúa como puente tecnológico hacia la biblioteca de JavaScript Leaflet, lo cual permite renderizar mapas de calor y representar geográficamente la red de sensores. Su uso es clave para analizar la disponibilidad de plazas en función de su proximidad a Puntos de Interés (POIs) específicos, como centros comerciales o áreas de ocio.

2.4.2 Quarto

Para democratizar el acceso a los resultados y la metodología del proyecto, se ha empleado Quarto. Se trata de un sistema de publicación científica y técnica de código abierto, construido sobre la herramienta conversora universal Pandoc. Quarto permite entrelazar texto narrativo en formato *Markdown* con código

ejecutable de Python, para generar documentos dinámicos de alta calidad estética [2].

En este contexto, Quarto se ha utilizado para el desarrollo y despliegue del entorno web estructurado del proyecto. Esta tecnología facilita la compilación de un *portfolio* interactivo que agrupa la arquitectura de datos, las visualizaciones generadas por Grafana y Folium, y las conclusiones extraídas de los modelos predictivos, lo que ofrece una experiencia divulgativa alternativa a la tradicional memoria estática en PDF.

3 Diseño e implementación

El presente capítulo constituye el núcleo técnico de este trabajo. En él se describe de manera exhaustiva el proceso de desarrollo de la plataforma, detallando las decisiones de diseño, el tratamiento de la información y la construcción del *pipeline* predictivo. El objetivo es proporcionar una visión clara y reproducible de cómo se han integrado las diferentes tecnologías para dar solución al problema del pronóstico de aparcamiento mediante *Deep Learning*.

3.1 Arquitectura general

Para garantizar la escalabilidad y robustez del sistema, se ha diseñado una arquitectura orientada a los datos, dividida en fases secuenciales que abarcan desde la ingesta hasta la visualización final. La estructura global del proyecto se ilustra en la Figura 3.1.

Tal y como se observa en la Figura 3.1, el sistema se articula de forma secuencial, dividiendo sus primeras fases (Capas 1 y 2) en dos flujos de trabajo diferenciados según la naturaleza temporal de los datos:

- **Flujo histórico.** Ejecutado de forma local mediante *scripts* de Python, se encarga de la extracción de datos desde el portal de *Open Data* de Melbourne y posteriores transformaciones. Estos datos serán empleados en entrenar el modelo de PyPOTS.
- **Flujo en tiempo real.** Orquestado íntegramente por Apache Airflow, interactúa de forma periódica con la API del Ayuntamiento para obtener los datos en tiempo real, estandarizarlos y mantener el sistema actualizado mediante el almacenamiento en DuckDB. Estos datos serán destinados para realizar la inferencia una vez el modelo ha sido entrenado.
- **Motor de modelado predictivo.** Realiza el entrenamiento con los datos históricos y ejecutar la inferencia para generar las predicciones futuras.

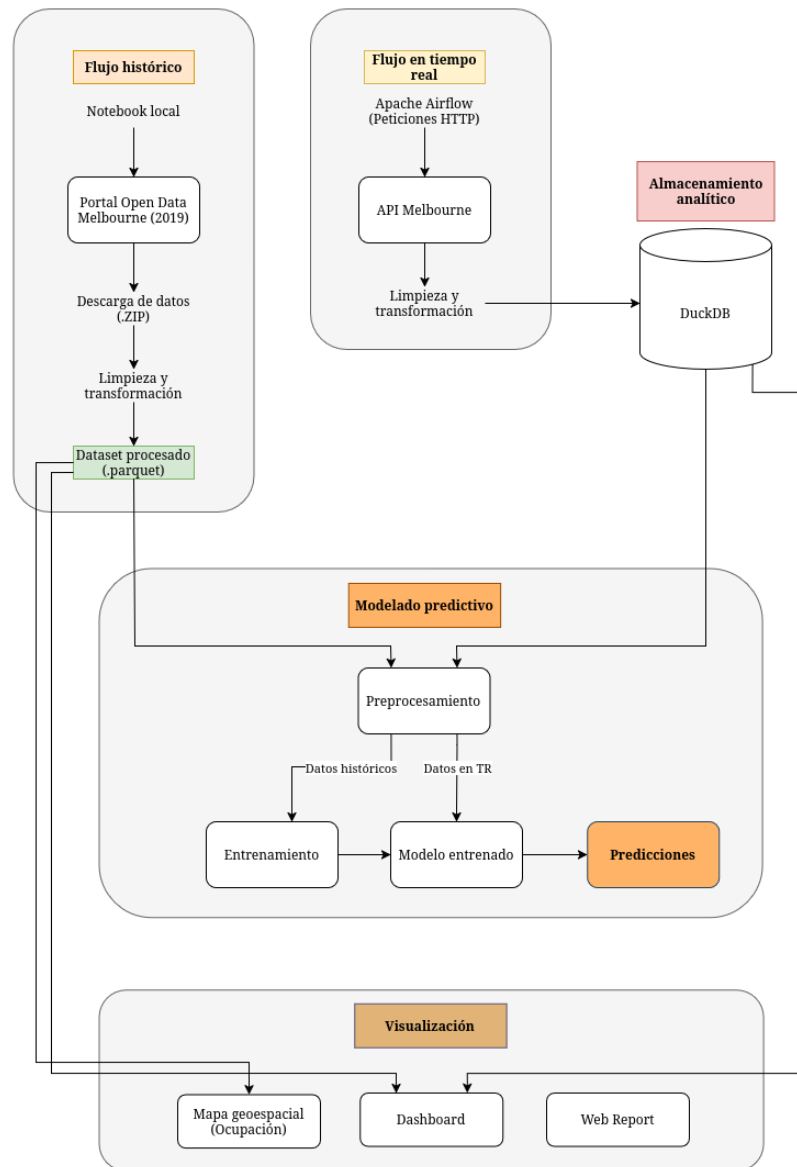


Figura 3.1: Arquitectura general del pipeline del sistema.

- **Interfaz de consumo y visualización.** Integración de herramientas específicas según el caso de uso: Grafana para la monitorización de métricas y predicciones, Folium para la representación interactiva de la ocupación geográfica, y Quarto para la web del proyecto

3.2 Origen y Comprensión de los Datos

El primer paso crítico en cualquier proyecto de analítica predictiva es la adquisición y comprensión del dominio de los datos. Para este trabajo, se ha tomado como base empírica el portal de *Open Data* del Ayuntamiento de Melbourne.

3.2.1 Justificación del *dataset* histórico. Extracción automatizada

Para la fase de entrenamiento y experimentación inicial, se recurrió al *dataset* histórico estático correspondiente al año 2019. La elección de este año en particular responde a una limitación operativa y contextual importante: debido a las anomalías de movilidad y a los cambios en las políticas de retención de datos durante la pandemia de la COVID-19, los conjuntos de datos posteriores a 2020 presentan vacíos de información o no fueron almacenados de forma persistente. Por tanto, 2019 representa el último ciclo temporal completo y fidedigno de la movilidad urbana ordinaria (incluyendo estacionalidad anual, festividades locales y variaciones estacionales).

Para garantizar la reproducibilidad del entorno de desarrollo y evitar el manejo manual de archivos masivos, la obtención de este conjunto de datos se ha procedimentado mediante un *pipeline* de extracción automatizado en Python. Haciendo uso de librerías nativas como *requests* para peticiones HTTP, se orquestó la conexión al repositorio original para la descarga del archivo comprimido por bloques (*chunks*) para evitar colapsar la memoria RAM.

Una vez finalizada la transferencia, el sistema localiza el archivo (*.zip*), extrae su contenido programáticamente utilizando la librería *zipfile* y almacena los datos tabulares en formato CSV para su posterior procesamiento.

3.2.2 Estructura del conjunto de datos

Este conjunto de datos masivo, compuesto por más de 42 millones de registros anuales, presenta un paradigma de almacenamiento orientado a eventos. Cada fila del documento no representa el estado general o agregado de una calle en un intervalo de tiempo dado, sino un evento asíncrono e individual: la llegada (*ArrivalTime*) o la salida (*DepartureTime*) física de un vehículo en un sensor o plaza de aparcamiento específica (*DeviceId* o *Bay ID*), acompañado de su marca temporal exacta.

Esta alta granularidad, si bien es excelente para capturar la realidad del tráfico de agitación plaza por plaza, requiere de un procesamiento intensivo para transformar un registro de eventos asíncronos en una serie temporal regular y estructurada con la que posteriormente poder modelar.

3.3 Análisis Exploratorio de Datos (EDA)

Antes de aplicar cualquier técnica de modelado o de diseñar el esquema de la base de datos analítica, es imperativo realizar un Análisis Exploratorio de Datos (EDA). Esta fase persigue un doble objetivo: comprender las distribuciones subyacentes de los datos de movilidad en Melbourne, así como las variables de interés, e identificar anomalías en las mediciones físicas de los sensores.

Debido al gran volumen de información, las fases iniciales de diseño del algoritmo de limpieza se prototiparon sobre una muestra representativa de 500.000 filas, un tamaño significativo para obtener conclusiones generalizables al conjunto completo de datos. Esta estrategia permitió agilizar las iteraciones de desarrollo y optimizar el consumo de memoria RAM antes de escalar el proceso al clúster completo.

Tras visualizar las observaciones del *dataset* en formato *DataFrame* de *Pandas*, se realizó una interpretación a alto nivel de las variables presentes, clasificándolas en cuatro categorías principales:

- **Variables temporales:**
 - *ArrivalTime* (**str**): Fecha y hora exacta en la que el sensor detectó la llegada del vehículo.

- *DepartureTime* (**str**): Fecha y hora exacta en la que el vehículo abandonó la plaza.
- *DurationMinutes* (**str**): Tiempo total de estacionamiento (diferencia temporal entre la llegada y la salida).
- **Variables de estado:**
 - *VehiclePresent* (**bool**): Variable objetivo (*target*). Indica la presencia física de un vehículo sobre el sensor durante ese periodo.
 - *InViolation* (**bool**): Indica si el vehículo excedió el tiempo permitido, incurriendo en una infracción.
 - *Sign* (**str**) / *SignPlateID* (**float64**): Texto y código identificador de la señal física de tráfico de la calle. Estas variables presentan una alta proporción de valores nulos.
- **Identificadores de plazas:**
 - *DeviceId* (**int64**): Número de serie electrónico del sensor.
 - *BayId* (**int64**): Identificador de la plaza de aparcamiento.
 - *StreetMarker* (**str**): Código alfanumérico identificativo. La tupla formada por (*DeviceId*, *StreetMarker*) constituye una clave única.
- **Variables de ubicación:**
 - *AreaName* (**str**): Barrio o distrito general al que pertenece la plaza.
 - *StreetName* (**str**) / *StreetId* (**int64**): Nombre y código numérico de la calle principal.
 - *BetweenStreet1* (**str**) / *BetweenStreet1ID* (**int64**): Primera calle transversal que acota el tramo de estacionamiento.
 - *BetweenStreet2* (**str**) / *BetweenStreet2ID* (**int64**): Segunda calle transversal que acota el tramo.
 - *SideName* (**str**) / *SideOfStreetCode* (**str**) / *SideOfStreet* (**int64**): Identificadores de la acera de estacionamiento y su ubicación geográfica (Norte, Sur, Este, Oeste). Existe una equivalencia predefinida entre códigos (1-C, 2-E, 3-N, 4-S, 5-W).

3.3.1 Transformación y tipificado de variables

Los datos brutos presentaban inconsistencias de formato que requirieron una estandarización previa. En primer lugar, la variable de duración (*DurationMinutes*) fue convertida de cadena de texto a valor numérico de coma flotante (**float**). Asimismo, las marcas de tiempo (*ArrivalTime* y *DepartureTime*), almacenadas originalmente como texto plano, se transformaron a objetos *datetime* nativos, lo cual es indispensable para habilitar la correcta indexación temporal y el posterior remuestreo de la serie.

3.3.2 Ingeniería de características (*Feature Engineering*)

Con las variables temporales correctamente tipificadas, se procedió a la descomposición de las fechas para extraer componentes estacionales subyacentes. A partir de la fecha de llegada, se generaron tres nuevas características fundamentales:

- **Hora del día (*Hour*):** busca capturar los picos de tráfico laboral diario.
- **Día de la semana (*WeekDay*):** busca entender el comportamiento de la movilidad entre días laborables y fines de semana.
- **Mes del año (*Month*):** ayuda a modelar la macro-estacionalidad, como periodos vacacionales o cambios de estación.

3.3.3 Análisis descriptivo y visualización

Una vez estandarizado el conjunto de datos y generadas las nuevas variables, se procedió a realizar un análisis visual descriptivo para obtener *insights* sobre el comportamiento de la movilidad.

En primer lugar, se analizó la distribución de la variable objetivo binaria (*VehiclePresent*), la cual indica la ocupación física de las plazas. Este paso es crítico para evaluar el balanceo de clases antes de la fase de modelado.

Como se aprecia en la Figura 3.2, se evaluó la proporción de registros correspondientes a plazas ocupadas frente a plazas libres. En problemas de *Deep Learning*, un desbalanceo severo en la variable a predecir podría inducir a la red neuronal a apostar sistemáticamente por la clase mayoritaria, reduciendo drásticamente su

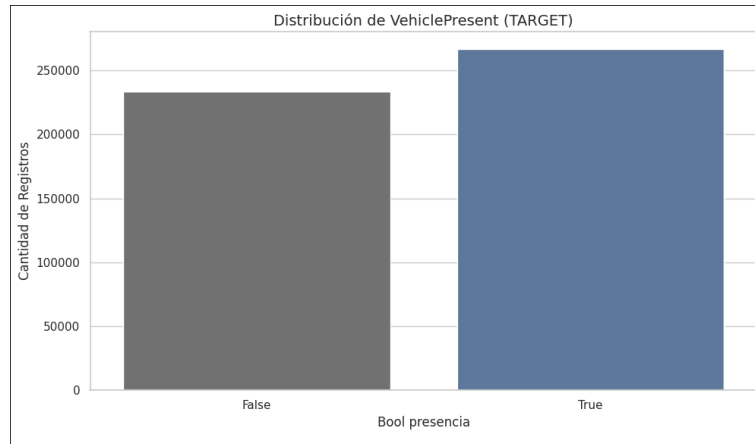


Figura 3.2: Distribución de la variable objetivo (*VehiclePresent*).



Figura 3.3: Distribución del volumen de eventos de estacionamiento según la hora del día.

capacidad de generalización. La visualización de esta distribución permite confirmar que no hay un desbalanceo notable, pues ambas clases presentan una representación equilibrada.

Posteriormente, se analizó el volumen de registros de aparcamiento en función de la hora del día, con el fin de identificar los patrones de rotación diarios.

Tal y como se ilustra en la Figura 3.3, la movilidad urbana en las plazas sensorizadas presenta un comportamiento fuertemente cíclico y dependiente del horario laboral. Se observa un valle profundo de inactividad durante la madrugada, seguido de un incremento pronunciado a partir de las 08:00 horas. El volumen de eventos se mantiene en niveles máximos durante las horas centrales del día, reflejando la alta rotación comercial y laboral, para finalmente descender de forma paulatina a partir de las 18:00 horas aproximadamente.

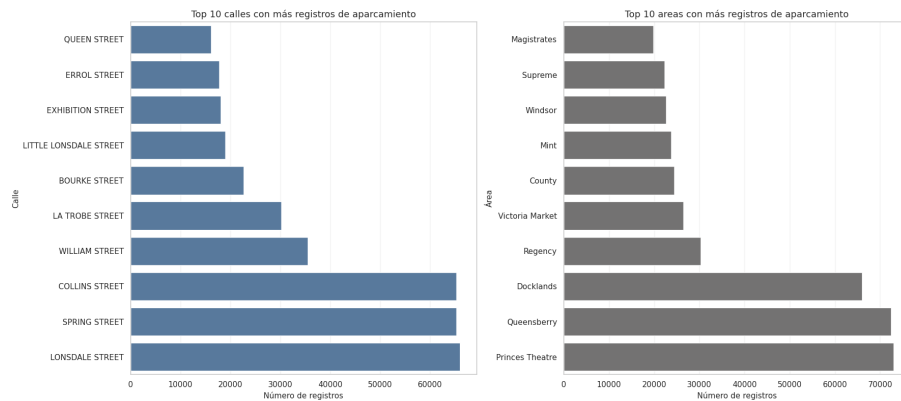


Figura 3.4: Top de vías o zonas con mayor densidad de registros de aparcamiento (*hotspots*).

Por último, se evaluó la distribución espacial de estos eventos para identificar los principales puntos de congestión o *hotspots* de la ciudad.

El análisis geográfico representado en la Figura 3.4 revela que la demanda de aparcamiento no se distribuye de manera uniforme, sino que se concentra asimétricamente en unas pocas vías principales. Estas calles, que acumulan la mayor densidad de registros, pueden corresponder a arterias de algún distrito central de negocios o a zonas con alta concentración de servicios. Esta información es importante, pues estas zonas serán áreas prioritarias para el modelo predictivo.

3.3.4 Selección de características

Tras entender las variables en profundidad, se llevó a cabo un proceso selección de las mismas (*Feature Selection*) para eliminar redundancias y evitar la fuga de información (*data leakage*) durante el entrenamiento. Se descartaron columnas descriptivas que representaban la misma entidad a nivel lógico, por ejemplo, identificadores internos del sensor frente a nombres de la calle y se conservaron las variables que componen el conjunto mínimo viable de características predictivas: *DeviceId*, *StreetId*, *VehiclePresent*, *ArrivalTime*, *DepartureTime*, así como el resto de variables temporales (*Hour*, *WeekDay*, *Month*).

Como paso final de esta fase exploratoria, el *DataFrame* limpio y preprocesado fue exportado en formato Parquet (*.parquet*). La elección de este formato de almacenamiento en columnas, en lugar del tradicional CSV, se implementó por su

Nuevo tamaño del dataset: (640875, 38)

DeviceId	Timestamp	VehiclePresent	StreetId	Hour_0	Hour_1	Hour_10	Hour_11	Hour_12	Hour_13	...	WeekDay_0	WeekDay_1	WeekDay_2	WeekDay_3	WeekDay_4	WeekDay_5	WeekDay_6	Month_1	Month_2	Month_3	
0	23914	2019-01-01	NaN	528.0	1	0	0	0	0	...	0	1	0	0	0	0	0	0	1	0	0
1	23915	2019-01-01	NaN	123.0	1	0	0	0	0	...	0	1	0	0	0	0	0	0	1	0	0
2	23916	2019-01-01	False	123.0	1	0	0	0	0	...	0	1	0	0	0	0	0	0	1	0	0
3	23917	2019-01-01	True	894.0	1	0	0	0	0	...	0	1	0	0	0	0	0	0	1	0	0
4	23918	2019-01-01	True	926.0	1	0	0	0	0	...	0	1	0	0	0	0	0	0	1	0	0

Figura 3.5: Transformación de variables temporales mediante *One-Hot Encoding* en el *dataset* final preprocesado.

significativa reducción en los tiempos de lectura y escritura en disco para las fases posteriores.

3.4 Adaptación temporal para *Deep Learning*

3.4.1 Regularización de la serie (*Grid*)

Como se mencionó en la subsección 3.2.2, los datos originales son asíncronos y transaccionales. Sin embargo, las arquitecturas de *Deep Learning* para series temporales requieren secuencias de longitud fija e intervalos regulares. Para solventar esta discrepancia, se implementó un algoritmo de agrupamiento y remuestreo que transforma los eventos asíncronos individuales en un sistema de cuadrícula (*Grid*). Este enfoque discretiza el tiempo en intervalos fijos de 15 minutos y recalcula el estado de ocupación de cada plaza (*DeviceId*) para cada uno de dichos intervalos.

3.4.2 Codificación categórica posicional (*One-Hot Encoding*)

Las variables temporales extraídas previamente en la fase de EDA (*Hour*, *WeekDay*, *Month*) poseen una naturaleza cíclica y categórica. Si estas variables se introdujeran en la red neuronal como valores enteros secuenciales (por ejemplo, Lunes = 1, Martes = 2), el modelo podría inferir erróneamente una relación matemática de magnitud u orden jerárquico que no existe en la realidad.

Para neutralizar este riesgo sobre la cuadrícula temporal ya regularizada, se aplicó una técnica de codificación posicional conocida como *One-Hot Encoding*.

Tal y como ilustra la Figura 3.5, esta técnica expande cada característica categórica en múltiples columnas binarias. Como resultado, la topología final del *dataset* incrementa su dimensionalidad horizontal, pero garantiza que la red neuronal pondere correctamente los patrones estacionales sin sesgos aritméticos.

Como resultado del procesamiento previo, se obtiene el conjunto de datos formalmente preparado para el proceso de entrenamiento.

4 Experimentos y validación

Atención: Este capítulo se introdujo como requisito en 2019.

Describe los experimentos y casos de test que tuviste que implementar para validar tus resultados. Incluye también los resultados de validación que permiten afirmar que tus resultados son correctos.

4.1 Incorporación de código en la memoria

Es bastante habitual que se reproduzcan fragmentos de código en la memoria de un TFG/TFM. Esto permite explicar detalladamente partes del desarrollo que se ha realizado que se consideren de especial interés. No obstante, tampoco es conveniente pasarse e incluir demasiado código en la memoria, puesto que se puede alargar mucho el documento. Un recurso muy habitual es subir todo el código a un repositorio de un servicio de control de versiones como GitHub o GitLab, y luego incluir en la memoria la URL que enlace a dicho repositorio.

Para incluir fragmentos de código en un documento $\text{\LaTeX}$ se pueden combinar varias herramientas:

- El entorno `\begin{listing}[]...\end{listing}` permite crear un marco en el que situar el fragmento de código (parecido al generado cuando insertamos una tabla o una figura). Podemos insertar también una descripción (*caption*) y una etiqueta para referenciarlo luego en el texto.
- Dentro de este entorno, se puede utilizar el paquete `minted`¹, que utiliza el paquete Python Pygments para resaltado de sintaxis (coloreando el código). Como se puede ver en el siguiente ejemplo, hay muchas opciones de

¹https://es.overleaf.com/learn/latex/Code_Highlighting_with_minted

configuración que permiten controlar cómo se va a mostrar el código (incluir números de línea, saltos de línea, tamaño y tipo de fuente, espaciado, código de colores para resaltado, etc.).

```
# A dictionary is built to define the data type contained
↳ by each column
dtype_scheme = {'budget': np.int64, 'genres': np.object,
↳ 'homepage': np.str, 'id': np.int64, 'keywords':
↳ np.object, 'original_language': np.str,
↳ 'original_title': np.str, 'overview': np.str,
↳ 'popularity': np.float64, 'production_companies':
↳ np.object, 'production_countries': np.object,
↳ 'release_date': np.object, 'revenue': np.int64,
↳ 'runtime': np.float64, 'spoken_languages':
↳ np.object, 'status': np.object, 'tagline': np.str,
↳ 'title': np.str, 'vote_average': np.float64,
↳ 'vote_count': np.int64}

# When loading the data from the .csv file, we provide
↳ the scheme to be followed for data typing
df1 = dd.read_csv('tmdb_5000_movies.csv',
↳ dtype=dtype_scheme)
```

Código 4.1: Lectura de un fichero *.csv y tipado de datos.

Otra ventaja del entorno `listing` es que se puede generar automáticamente un índice (con entradas hiperenlazadas) de fragmentos de código, para incluirlo al comienzo del documento junto con los índices de figuras, tablas, etc.

4.1.1 Fuentes monoespaciadas

A veces se incluyen nombres de archivos, paquetes, etc. como texto monoespaciado, utilizando el comando `LATEX\texttt{}`. Sin embargo, esto puede generar un problema cuando las palabras en fuente monoespaciada alcanzan el final de una línea. En ese caso, el compilador rehusa muchas veces romper la palabra y deja la línea demasiado larga respecto al resto.

Para evitar esto, especialmente en párrafos más cortos de lo habitual (como en una lista no numerada), se puede utilizar el entorno:

```
\begin{sloppypar}
...
\end{sloppypar}
```

Se muestra a continuación con un ejemplo real.

- Los valores contenidos en las columnas `genres`, `spoken_languages`, `production_companies` y `production_countries`, clasificados originalmente como `np.objects`, se corresponden en realidad con listas de objetos JSON que han sido almacenadas como cadenas de caracteres. A través de la función `get_values(obj, key)` definida específicamente para ello, se transformará dicha cadena de caracteres en una lista de diccionarios a través de la función `json.loads(obj)` y se devolverá una tupla que recopile los valores de los mismos para la clave indicada, un objeto de Python mucho más manejable de cara a realizar consultas sobre el *dataset*.

4.2 Contenido matemático y ecuaciones

El contenido matemático se puede escribir en línea utilizando los delimitadores `\(...\)`, o bien delimitado por dos símbolos de \$. Actualmente, se prefiere la primera opción para favorecer la compatibilidad con otras herramientas como Pandoc, así como para aprovechar el código para escribir ecuaciones con motores de interpretación de contenido matemático en HTML.

Si queremos que el contenido matemático se muestra en un bloque aparte, con una notación que ocupe más espacio vertical (para sumatorios, integrales, fracciones, etc.), entonces debemos usar el llamado *display mode*, delimitado por `\[...\]`, o bien por dos símbolos \$\$\$. De nuevo, la primera opción es preferible.

Por ejemplo, para mostrar en la misma línea una ecuación de segundo grado, escribimos $3x^2 + 2x + 1 = 0$. Si preferimos el contenido en un bloque independiente:

$$F(x) = \int_b^a \frac{1}{3}x^3$$

$$\frac{1}{\sqrt{x}}$$

$$\begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$$

Ahora bien, en un documento en el que tenemos que hacer referencia a las ecuaciones dentro del texto de explicación, es conveniente otorgarle a cada bloque matemático un identificador numérico, similar al que usamos en figuras y tablas. Para ello, empleamos el bloque `\begin{equation} contenidos... \end{equation}`. En esta ocasión ya no tenemos que incluir los delimitadores de contenido matemático en *display mode*.

$$\frac{1}{\sqrt{x}} \quad (4.1)$$

Si añadimos una etiqueta a la ecuación, entonces podemos referenciar la Ecuación 4.1 en el texto. Veamos más ejemplos con caracteres especiales.

$$\hat{f}(\omega) = \frac{1}{2\pi} \sum_{-\infty}^{\infty} f(x) e^{-i\omega x} dx, \quad (4.2)$$

$$\tilde{f}(\omega) = \frac{1}{2\pi} \int_{-\infty}^{\infty} f(x) e^{-i\omega x} dx, \quad (4.3)$$

$$\dot{\vec{\omega}} = \vec{r} \times \vec{I}. \quad (4.4)$$

4.2.1 Ecuaciones en múltiples líneas y elementos alineados

Algunos ejemplos que muestran alineamiento de ecuaciones en diferentes líneas con el entorno `\begin{align*} ... \end{align*}`.

$x = y$	$w = z$	$a = b + c$
$2x = -y$	$3w = \frac{1}{2}z$	$a = b$
$-4 + 5x = 2 + y$	$w + 2 = -1 + w$	$ab = cb$

Nota: las ecuaciones anteriores en varias líneas tienen un modo de escritura matemática definido para cada línea, no globalmente (a nivel de ecuación).

$$\begin{aligned}f(x) &= x^2 \\g(x) &= \frac{1}{x} \\F(x) &= \int_b^a \frac{1}{3}x^3\end{aligned}$$

4.2.2 Teoremas y conjuntos

Teorema 1 Para todo número entero no negativo n , tenemos:

$$(1+x)^n = \sum_{i=0}^n \binom{n}{i} x^i.$$

La expansión en series de Taylor de la función e^x viene dada por:

$$e^x = 1 + x + \frac{x^2}{2} + \frac{x^3}{6} + \dots = \sum_{n \geq 0} \frac{x^n}{n!}. \quad (4.5)$$

$$\forall x \in X, \quad \exists y \leq \epsilon,$$

$$\frac{n!}{k!(n-k)!} = \binom{n}{k}.$$

Teorema 2 Para cualesquiera conjuntos A , B y C , se cumple que:

$$(A \cup B) - (C - A) = A \cup (B - C).$$

5 Conclusiones y trabajos futuros

5.1 Consecución de objetivos

Esta sección es la sección espejo de las dos primeras del capítulo de objetivos, donde se planteaba el objetivo general y se elaboraban los específicos.

Es aquí donde hay que debatir qué se ha conseguido y qué no. Cuando algo no se ha conseguido, se ha de justificar, en términos de qué problemas se han encontrado y qué medidas se han tomado para mitigar esos problemas.

Y si has llegado hasta aquí, siempre es bueno pasarle el corrector ortográfico, que las erratas quedan fatal en la memoria final. Para eso, en Linux tenemos *aspell*, que se ejecuta de la siguiente manera desde la línea de *shell*:

```
aspell --lang=es_ES -c memoria.tex
```

5.2 Aplicación de lo aprendido

Aquí viene lo que has aprendido durante el Grado/Máster y que has aplicado en el TFG/TFM. Una buena idea es poner las asignaturas más relacionadas y comentar en un párrafo los conocimientos y habilidades puestos en práctica.

1. a
2. b

5.3 Lecciones aprendidas

Aquí viene lo que has aprendido en el Trabajo Fin de Grado/Máster.

1. Aquí viene uno.
2. Aquí viene otro.

5.4 Trabajos futuros

Ningún proyecto ni software se termina, así que aquí vienen ideas y funcionalidades que estaría bien tener implementadas en el futuro.

Es un apartado que sirve para dar ideas de cara a futuros TFGs/TFMs.

A Contenido adicional

A.1 Dimensiones de página

A continuación se incluye un resumen de las dimensiones calculadas para el diseño de maquetación de las páginas de la memoria.

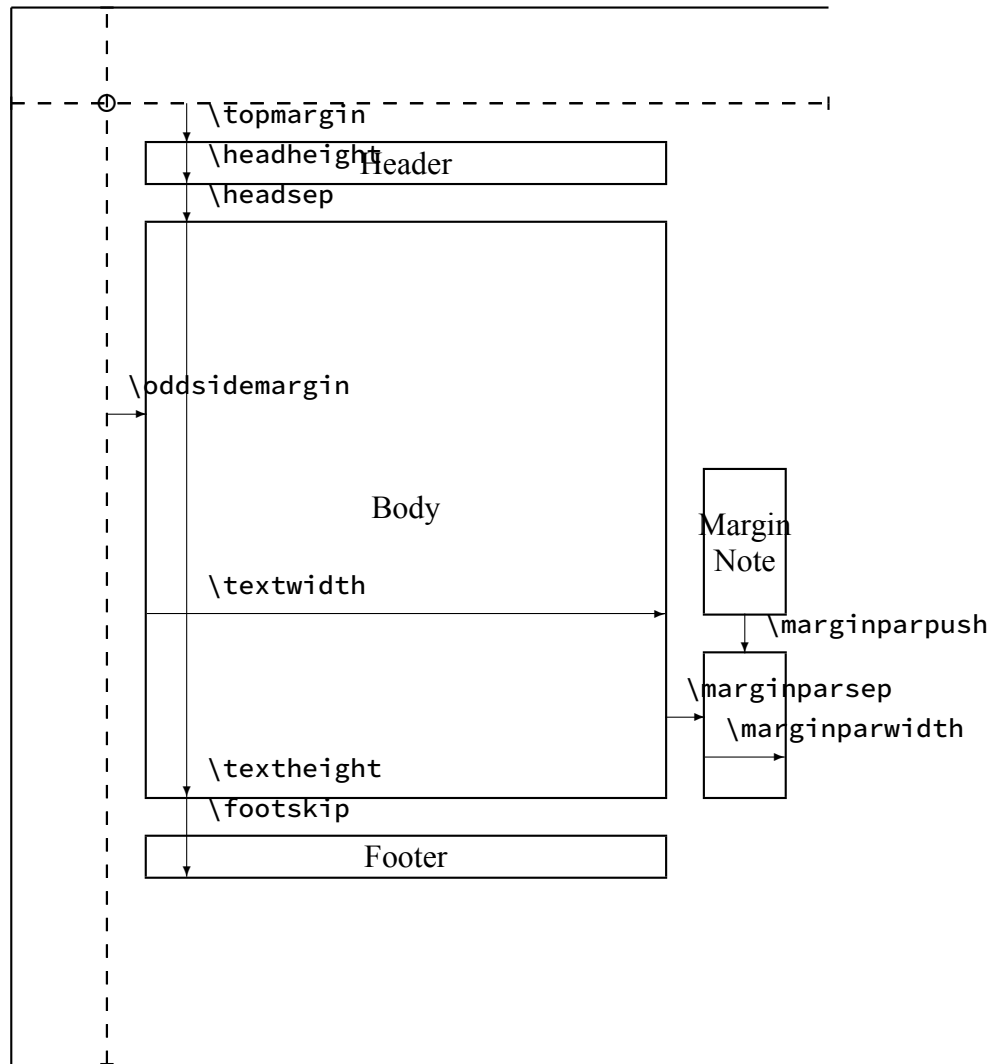
Actual page layout values.

<code>\paperheight = 296.99655mm</code>	<code>\paperwidth = 209.99756mm</code>
<code>\hoffset = 0mm</code>	<code>\voffset = 0mm</code>
<code>\evensidemargin = 14.1124mm</code>	<code>\oddsidemargin = 14.1124mm</code>
<code>\topmargin = -10.2513mm</code>	<code>\headheight = 5.97475mm</code>
<code>\headsep = 7.64415mm</code>	<code>\textheight = 220.09608mm</code>
<code>\textwidth = 135.97327mm</code>	<code>\footskip = 17.83636mm</code>
<code>\marginparsep = 4.51273mm</code>	<code>\marginparpush = 1.40582mm</code>
<code>\columnsep = 3.51456mm</code>	<code>\columnseprule = 0mm</code>
<code>lem = 4.21747mm</code>	<code>lex = 1.88632mm</code>

`\marginparwidth: 23.0081mm`

`\marginparwidth: 65.4652pt`

The circle is at 1 inch from the top and left of the page. Dashed lines represent (`\hoffset + 1 inch`) and (`\voffset + 1 inch`) from the top and left of the page.



Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales

commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

Referencias

- [1] Amir H Alavi et al. «Internet of Things-enabled smart cities: State-of-the-art and future trends». En: *Measurement* 129 (2018), págs. 589-606.
- [2] J.J. Allaire et al. *Quarto*. Posit Software. 2022.
URL: <https://github.com/quarto-dev/quarto-cli>
(visitado 20-05-2024).
- [3] Wenjie Du, David Cote y Tianchun Liu. «PyPOTS: a Python toolbox for data mining on partially observed time series». En: *arXiv preprint arXiv:2305.10370* (2023).
- [4] Mohammed Ali Al-Garadi et al. «A survey of machine and deep learning methods for internet of things (IoT) security». En: *IEEE Communications Surveys & Tutorials* 22.3 (2020), págs. 1646-1685.
- [5] Trista Lin, Hervé Rivano y Frédéric Le Mouel. «A survey of smart parking solutions». En: *IEEE Transactions on Intelligent Transportation Systems* 18.12 (2017), págs. 3229-3253.
- [6] Mark Raasveldt y Hannes Mühleisen. «DuckDB: an Embeddable Analytical Database». En: *Proceedings of the 2019 International Conference on Management of Data*. ACM. 2019, págs. 1981-1984.
- [7] Donald C Shoup. «Cruising for parking». En: *Transport Policy* 13.6 (2006), págs. 479-486.
- [8] Guido Van Rossum y Fred L Drake Jr. *Python tutorial*. Centrum voor Wiskunde en Informatica Amsterdam, The Netherlands, 1995.